

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana

w specjalności TO-DO

Bot Bowl - nauka sieci neuronowej gry w grę Blood Bowl

inż. Mikołaj Klębowski

numer albumu 299253

promotor

dr inż. Krzysztof Hryniów

WARSZAWA 2025

Bot Bowl - nauka sieci neuronowej gry w grę Blood Bowl

Streszczenie

To jest streszczenie. To jest trochę za krótkie, jako że powinno zająć całą stronę.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Słowa kluczowe: A, B, C

Bot Bowl - learning the neural network game of Blood Bowl

Abstract

This is abstract. This one is a little too short as it should occupy the whole page.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Keywords: X, Y, Z

Spis treści

1	Wstęp	9
2	Sieci Neuronowe	11
2.1	Funkcja aktywacji	12
2.2	Architektura sieci neuronowej	14
2.3	Techniki uczenia sieci neuronowych	16
2.4	Funkcje kosztu (straty)	18
2.5	Algorytmy optymalizacji	19
2.6	Problemy gradientowe i usprawnienia	22
3	Sieci Konwolucyjne	25
3.1	Geneza	25
3.2	Sposób działania	25
3.2.1	Warstwy wspierające	27
4	Środowisko testowe	29
4.1	Just in case	30
	Bibliografia	31
	Wykaz skrótów i symboli	31
	Spis rysunków	33
	Spis tabel	35

Rozdział 1

Wstęp

W ciągu ostatnich lat sztuczna inteligencja przekształciła wiele obszarów gospodarki i życia codziennego. Systemy oparte na sieciach neuronowych wspierają wyszukiwarki internetowe, umożliwiają autonomiczną jazdę samochodów i optymalizują procesy produkcyjne. Rosnąca zdolność tych modeli do analizowania ogromnych zbiorów danych oraz uczenia się z nich pozwala na podejmowanie decyzji bez bezpośredniego nadzoru człowieka. Przykładowo, w sektorze finansowym sieci neuronowe zarządzają portfelem inwestycyjnym, prognozowanie ryzyka kredytowego czy dynamiczne kształtowanie cen.

Jednak aby w pełni ocenić możliwości i ograniczenia poszczególnych architektur sieci, niezbędne są środowiska testowe o jasno zdefiniowanych regułach, dużej złożoności stanów oraz szerokim wachlarzu możliwych działań. Gry od lat są używane jako środowisko do testowania algorytmów uczenia maszynowego. Dzięki mierzalnym wynikom rozgrywek oraz możliwości powtarzalnego odtwarzania scenariuszy, można precyzyjnie porównywać efektywność różnych metod – od prostych wielowarstwowych perceptronów, przez konwolucyjne sieci neuronowe, aż po nowoczesne modele hybrydowe.

W ramach tej pracy zostanie przeprowadzone porównanie różnych rozwiązań w grze bloodbowl. Ze względu na wysoki branching factor, losowość rzutów kośćmi oraz wieloetapowy charakter rozgrywki, Bloodbowl stanowi doskonałe środowisko do badania procesów decyzyjnych agentów uczonych ze wzmocnieniem. Celem niniejszej pracy jest porównanie wybranych architektur sieci neuronowych pod kątem szybkości konwergencji, stabilności uczenia oraz jakości wypracowanych strategii. W kolejnych rozdziałach przedstawione zostaną szczegóły środowiska BloodBowl, opis testowanych modeli oraz wyniki przeprowadzonych eksperymentów.

Rozdział 2

Sieci Neuronowe

Idea sztucznych sieci neuronowych narodziła się podczas prób modelowania pracy ludzkiego mózgu. Warren S. McCulloch oraz Walter Pitts w swojej pracy "A logical calculus of the ideas immanent in nervous activity" [McCulloch1943] zaproponowali model sieci neuronowej, jako grafu skierowanego, gdzie węzły to matematyczne modele neuronu takie, że

$$N_i(t+1) = H\left(\sum_{j=1}^n w_{ij}(t)N_j(t) - \Theta_i(t)\right)$$

gdzie:

- N - stan neuronu, może przyjąć wartość 0 lub 1
- H - funkcja skokowa Heaviside'a
- w_{ij} - waga połączenia między wektorem j oraz i .
- Θ - próg wyzwalania, neuron aktywuje się, gdy sygnał wejściowy go przekroczy.

Sieć złożona z takich neuronów jest nazywana dzisiaj perceptronem. Służy ona do zadań klasyfikacji binarnej. Potrafi określić przynależność badanego obiektu do jednej z klas na podstawie wybranych cech. W 1957 roku Frank Rosenblatt opracował pierwszy uczący się perceptron, który stanowił przełom w rozwoju sztucznych sieci neuronowych. Pomimo swojej innowacyjności, model ten miał istotne ograniczenia – nie potrafił rozwiązywać problemów nieliniowych, takich jak klasyczny przykład funkcji XOR, a jego struktura uniemożliwiała zastosowanie metod uczenia opartych na obliczaniu pochodnych. Źródłem tego ograniczenia była funkcja skokowa Heaviside'a, wykorzystywana jako funkcja aktywacji neuronu. Współcześnie zastępuje się ją funkcjami ciągłymi i różniczkowalnymi, takimi jak sigmoid, tangens hiperboliczny czy ReLU (Rectified Linear Unit) wraz z jej modyfikacjami (np. Leaky ReLU, Parametric ReLU), które umożliwiają skuteczne stosowanie algorytmów gradientowych, takich jak propagacja wsteczna błędów.

[minsky1969perceptrons]

Jak zostało wcześniej wspomniane ten model został stworzony w celu opisu sposobu działania biologicznego neuronu. Można go uogólnić, pozwalając stworzyć bardziej uniwersalny model matematyczny.

$$y = f \left(\sum_{i=1}^m w_i(t) x_i(t) + b \right)$$

gdzie:

- w_i – waga
- x_i – wejście,
- f – funkcja aktywacji,
- y – wyjście,
- m – liczba wejść,
- b – stała wartość (ang. bias).

Do funkcji aktywacji f przekazywana jest ważona suma sygnałów wejściowych, do której dodaje się opcjonalnie parametr bias. Taki opis umożliwia zastosowanie dowolnej funkcji aktywacji, co pozwala sieci neuronowej modelować nieliniowe zależności.

2.1 Funkcja aktywacji

Jednym z kluczowych elementów każdego neuronu sztucznego jest funkcja aktywacji, która decyduje o tym, w jaki sposób sygnał wejściowy zostanie przekształcony w sygnał wyjściowy. Od jej właściwości zależy zdolność sieci neuronowej do modelowania złożonych, nieliniowych zależności.

Pierwszym powszechnie stosowanym rozwiązaniem była funkcja skokowa Heaviside'a:

$$H(x) = \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

która aktywuje neuron tylko po przekroczeniu określonego progu. Funkcja ta dobrze oddaje ideę neuronu biologicznego, który „uaktywnia się” po przekroczeniu pewnego poziomu pobudzenia, jednak z matematycznego punktu widzenia ma istotną wadę — nie jest różniczkowalna. Jej pochodna jest równa zero dla wszystkich wartości oprócz punktu $x=0$, gdzie jest nieokreślona. W konsekwencji uniemożliwia to stosowanie metod uczenia opartych na gradientach, takich jak propagacja wsteczna błędów.

Aby umożliwić płynne i różniczkowalne przejście między stanami aktywacji, wprowadzono funkcje ciągłe, takie jak sigmoid i tangens hiperboliczny ($\tanh$). Funkcja sigmoidalna ma postać:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

a tangens hiperboliczny:

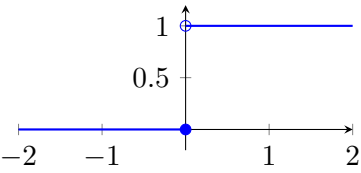
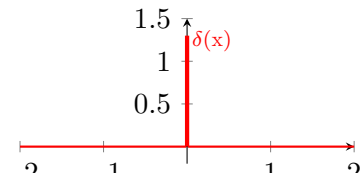
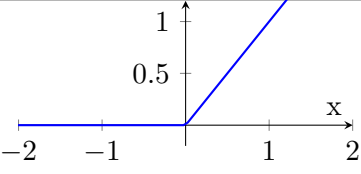
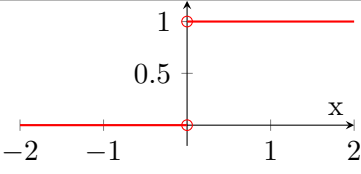
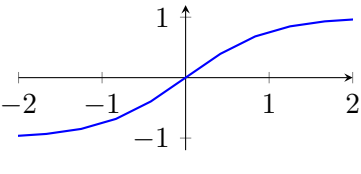
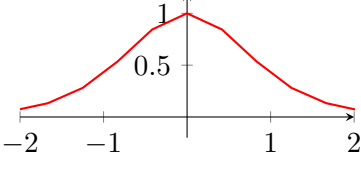
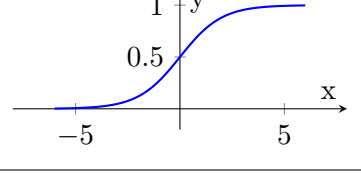
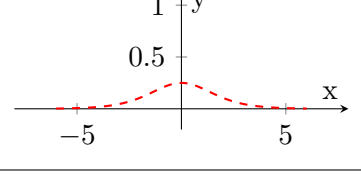
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Obie funkcje zapewniają gładką, różniczkowalną zależność między wejściem a wyjściem neuronu, dzięki czemu możliwe jest stosowanie gradientowych metod optymalizacji. Dodatkowo funkcja $\tanh(x)$ ma zakres $(-1, 1)$, co czyni ją bardziej symetryczną względem zera i ułatwia uczenie w sieciach głębokich. Ich wadą jest jednak zjawisko nasycenia – dla dużych wartości argumentu pochodna staje się bliska zeru, co prowadzi do problemu zanikającego gradientu.

W odpowiedzi na te ograniczenia opracowano funkcję ReLU (Rectified Linear Unit), definiowaną jako:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

Funkcja ta jest ciągła w całej swojej dziedzinie, również w punkcie $x=0$, lecz nie jest różniczkowalna w tym punkcie — lewa pochodna wynosi 0, a prawa 1. W praktyce jednak przyjmuje się, że funkcja jest „różniczkowalna prawie wszędzie”, co pozwala skutecznie stosować ją w uczeniu sieci metodami gradientowymi. ReLU eliminuje zjawisko zanikającego gradientu dla dodatnich wartości wejść, dzięki czemu znacząco przyspiesza uczenie głębokich modeli. Współczesne warianty, takie jak Leaky ReLU, Parametric ReLU czy ELU (Exponential Linear Unit), pozwalają dodatkowo ograniczyć problem tzw. martwych neuronów, które przestają reagować na sygnał uczący.

Porównanie funkcji i ich pochodnych	
Funkcja Heaviside $H(x)$	Pochodna $H(x)$
	
Funkcja ReLU(x)	Pochodna ReLU(x)
	
Tangens hiperboliczny $\tanh(x)$	Pochodna $\tanh(x)$
	
$\sigma(x)$	Pochodna $\sigma(x)$
	

2.2 Architektura sieci neuronowej

W sztucznej sieci neuronowej neurony są zorganizowane w warstwy, z których każda przetwarza sygnały z poprzedniej i przekazuje wynik dalej. Typowa struktura to warstwa wejściowa, jedna lub więcej warstw ukrytych oraz warstwa wyjściowa. Zazwyczaj warstwy są ze sobą w pełni połączone (ang. *fully connected*), co oznacza, że każdy neuron danej warstwy jest połączony z każdym neuronem warstwy następnej.

Z punktu widzenia matematycznego, każda warstwa sieci realizuje pewną funkcję przekształcającą wektor wejściowy w nową reprezentację:

$$h^{(l)} = f^{(l)} \left(W^{(l)} h^{(l-1)} + b^{(l)} \right), \quad (1)$$

gdzie:

- $h^{(l)}$ – wektor wyjść (aktywacji) z warstwy l ,
- $W^{(l)}$ – macierz wag warstwy l ,
- $b^{(l)}$ – wektor przesunięć (*bias*),

- $f^{(l)}(\cdot)$ – funkcja aktywacji danej warstwy.

Cała sieć neuronowa może być opisana jako złożenie (kompozycja) takich funkcji:

$$f(x) = f^{(L)}\left(f^{(L-1)}\left(\dots f^{(1)}(x) \dots\right)\right), \quad (2)$$

gdzie L oznacza liczbę warstw w sieci, a x jest wektorem wejściowym.

Głębokość i zdolność reprezentacyjna

Określenie *płytką sieć* (*shallow network*) odnosi się do modeli z jedną warstwą ukrytą, natomiast *głębokie sieci* (*deep networks*) zawierają wiele takich warstw. Głębsza architektura pozwala na hierarchiczne przekształcanie danych wejściowych — od prostych cech niskiego poziomu po coraz bardziej abstrakcyjne reprezentacje. Dzięki temu kolejne warstwy mogą uczyć się złożonych zależności, które trudno byłoby odwzorować w jednej transformacji.

Zależność pomiędzy głębokością sieci a jej zdolnością reprezentacyjną opisuje tzw. **twierdzenie o uniwersalnej aproksymacji**. Mówi ono, że już sieć z jedną warstwą ukrytą i wystarczającą liczbą neuronów jest w stanie przybliżyć dowolną funkcję ciągłą na zwartym zbiorze [Cybenko1989, HORN1989]. W praktyce jednak takie sieci wymagają ogromnej liczby neuronów i parametrów, co czyni je nieefektywnymi obliczeniowo. Sieci głębokie uzyskują podobną moc aproksymacyjną przy znacznie mniejszej liczbie parametrów, rozkładając złożoność obliczeń na wiele etapów przetwarzania.

Formalnie działanie sieci z jedną warstwą ukrytą można zapisać jako:

$$f(x) = \sum_{i=1}^N \alpha_i \sigma(w_i^\top x + b_i), \quad (3)$$

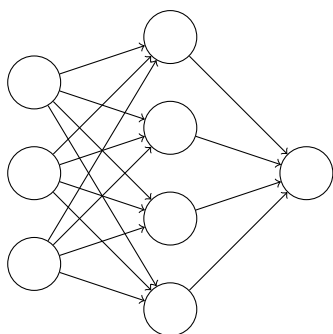
gdzie $\sigma(\cdot)$ jest nieliniową funkcją aktywacji, a α_i , w_i , b_i stanowią parametry uczone w procesie treningu.

Badania [RolnickTegmark2017, telgarsky2016benefits, H.N.Mhaskar2020] wykazały istnienie tzw. **hierarchii głębokości**, zgodnie z którą zwiększanie liczby warstw jest znacznie bardziej efektywne niż ich poszerzanie. Dla pewnych klas funkcji ograniczenie głębokości prowadzi do wykładniczego wzrostu liczby neuronów wymaganych do osiągnięcia tej samej jakości aproksymacji. Zależność tę można opisać asymptotycznie:

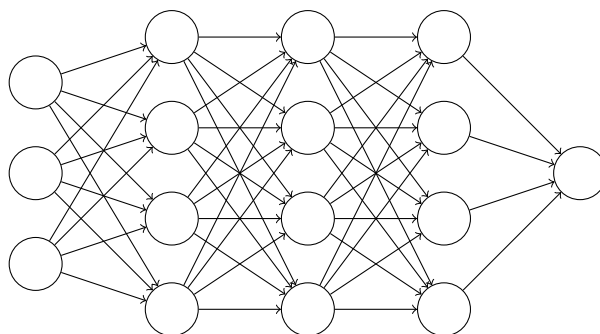
$$N_{\text{shallow}} = \mathcal{O}(2^k), \quad N_{\text{deep}} = \mathcal{O}(k^3), \quad (4)$$

gdzie k oznacza głębokość sieci. Wzrost głębokości modelu pozwala zatem osiągnąć tę samą moc reprezentacyjną przy istotnie mniejszej liczbie neuronów i połączeń.

Płytka sieć neuronowa



Głęboka sieć neuronowa



Rysunek 1. Porównanie płytkiej i głębokiej sieci neuronowej z wyrównanym położeniem neuronów wyjściowych.

2.3 Techniki uczenia sieci neuronowych

Uczenie sieci neuronowych to proces, podczas którego model dopasowuje swoje parametry w taki sposób, aby minimalizować błąd predykcji. Efektywność tego procesu decyduje o zdolności modelu do generalizacji, czyli poprawnego działania na podstawie danych, których nie widział podczas treningu. W praktyce, uczenie polega na iteracyjnym modyfikowaniu parametrów na podstawie informacji zwrotnej wyrażonej za pomocą funkcji kosztu (straty), której wartość wskazuje, jak bardzo przewidywania modelu odbiegają od pożądanych wyników.

Uczenie odbywa się iteracyjnie poprzez modyfikację wag modelu w kierunku przeciwnym do gradientu funkcji kosztu. Każdy cykl uczenia (epoka) obejmuje cztery główne etapy:

- Propagacja w przód (forward pass) – dane wejściowe przechodzą przez kolejne warstwy modelu, aż do uzyskania przewidywania $\hat{y}$.
- Obliczenie błędu (loss) – funkcja kosztu $(\cdot, \hat{y})$ mierzy różnicę między wartością przewidywaną a oczekiwaną.
- Propagacja wsteczna (backpropagation) – na podstawie reguły łańcuchowej obliczane są gradienty funkcji kosztu względem wag.
- Aktualizacja wag (update) – parametry są modyfikowane zgodnie z wybranym algorytmem optymalizacji, np. SGD lub Adamem.

Na podstawie sposobu pozyskiwania informacji zwrotnej i dostępności danych uczących wyróżnia się trzy główne paradygmaty uczenia: uczenie nadzorowane, uczenie nienadzorowane oraz uczenie ze wzmocnieniem. (Bishop, 2006)

Uczenie nadzorowane

Uczenie nadzorowane (*supervised learning*) jest najczęściej stosowanym paradygmatem uczenia sieci neuronowych. Zakłada ono, że dostępny jest zbiór danych uczących składający się z par (x, y) , gdzie x reprezentuje wektor cech opisujących przykład, a y jest przypisaną do niego etykietą lub wartością

wyjściową. Celem modelu jest nauczenie się odwzorowania $f: x \rightarrow y$ w taki sposób, aby przewidywane wyjście $\hat{y}$ było jak najbardziej zbliżone do wartości rzeczywistej y [Goodfellow-et-al-2016].

W ramach uczenia nadzorowanego wyróżnia się dwa główne typy zadań:

- **Klasyfikacja** – gdy etykiety y należą do skończonego zbioru klas, a model ma za zadanie przypisać dany przykład do jednej z nich. Przykłady zastosowań obejmują rozpoznawanie obrazów, klasyfikację dokumentów tekstowych czy diagnozowanie chorób na podstawie danych medycznych.
- **Regresja** – gdy etykieta y jest wartością liczbową, a celem jest przewidzenie wartości ciągłej. Do typowych przykładów należą prognozowanie cen, przewidywanie popytu czy estymacja parametrów fizycznych w eksperymentach naukowych.

Dzięki swojej uniwersalności, uczenie nadzorowane jest stosowane w szerokim zakresie dziedzin, od rozpoznawania mowy i analizy obrazu, przez systemy rekomendacyjne, aż po prognozowanie finansowe i modelowanie procesów przemysłowych.

Uczenie nienadzorowane

Uczenie nienadzorowane (*unsupervised learning*) obejmuje metody, w których model uczy się wyłącznie na podstawie danych wejściowych x , bez dostępu do odpowiadających im etykiet wynikowych. Celem jest odkrycie wewnętrznej struktury danych, zależności pomiędzy przykładami lub reprezentacji pozwalających na ich bardziej efektywne opisanie [Goodfellow-et-al-2016].

W ramach uczenia nienadzorowanego wyróżnić można kilka głównych typów zadań:

- **Grupowanie (clustering)** – podział zbioru danych na zbiory (klastry) w taki sposób, aby przykłady w tym samym klastrze były do siebie możliwie podobne, a między klastrami występowały wyraźne różnice. Przykładami algorytmów są *k-means*, DBSCAN czy sieci Kohonena (SOM) [Kohonen1990].
- **Redukcja wymiarów** – przekształcenie danych do przestrzeni o mniejszej liczbie wymiarów przy zachowaniu jak największej ilości istotnej informacji. Popularne metody to analiza głównych składowych (PCA) czy autoenkodery.
- **Uczenie reprezentacji** – poszukiwanie wewnętrznych opisów danych (tzw. reprezentacji ukrytych), które mogą ułatwić dalsze zadania, np. klasyfikację lub generowanie nowych przykładów. Przykładem są modele generatywne, takie jak *Variational Autoencoder* (VAE) czy *Generative Adversarial Networks* (GAN) [GoodfellowGAN2014].

Uczenie nienadzorowane znajduje zastosowanie m.in. w segmentacji obrazów, eksploracji danych, kompresji informacji, wyszukiwaniu podobnych obiektów, detekcji anomalii czy jako etap wstępny w uczeniu nadzorowanym (tzw. *pretraining*).

Uczenie ze wzmocnieniem

Uczenie ze wzmocnieniem (*reinforcement learning*, RL) to proces, w którym agent uczy się poprzez interakcję ze środowiskiem. W każdym kroku czasowym t agent obserwuje stan s_t , wybiera akcję a_t ,

otrzymuje nagrodę r_t i przechodzi do kolejnego stanu s_{t+1} . Celem jest maksymalizacja zdyskontowanej sumy nagród:

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

gdzie $\gamma \in (0, 1)$ to współczynnik dyskontowania, który określa znaczenie przyszłych nagród. Uczenie ze wzmocnieniem znajduje zastosowanie w problemach decyzyjnych – w tym w robotyce, sterowaniu i grach strategicznych, takich jak BloodBowl, Go czy StarCraft, gdzie sieci neuronowe pełnią rolę polityki decyzyjnej (policy network).

2.4 Funkcje kosztu (straty)

Funkcja kosztu (ang. *loss function*, *cost function*) pełni kluczową rolę w procesie uczenia sieci neuronowych, ponieważ stanowi ilościową miarę różnicy pomiędzy przewidywaniami modelu $\hat{y}$ a wartościami oczekiwanymi y . To właśnie na podstawie wartości funkcji kosztu algorytm optymalizacji oblicza gradienty, które następnie służą do modyfikacji wag modelu w kierunku minimalizacji błędu [Goodfellow-et-al-2016]. Dobór odpowiedniej funkcji straty jest ściśle związany z charakterem problemu i typem danych wyjściowych.

W praktyce można wyróżnić dwie główne grupy funkcji kosztu:

- **Funkcje kosztu dla klasyfikacji** – stosowane, gdy zadaniem modelu jest przypisanie przykładu do jednej z klas:
 - *Entropia krzyżowa (cross-entropy loss)* – powszechnie używana w zadaniach klasyfikacji wieloklasowej i binarnej. Funkcja ta silnie karze błędne przewidywania o wysokim poziomie pewności.
 - *Hinge loss* – często stosowana w klasyfikatorach typu SVM, promuje dużą marginesową separację pomiędzy klasami.
- **Funkcje kosztu dla regresji** – używane, gdy przewidywane są wartości ciągłe:
 - *Mean Squared Error (MSE)* – średnia wartość kwadratu różnic pomiędzy przewidywaniami a wartościami rzeczywistymi; silnie penalizuje duże błędy.
 - *Mean Absolute Error (MAE)* – średnia wartość bezwzględna błędów; mniej wrażliwa na wartości odstające niż MSE.

Dobór funkcji kosztu powinien uwzględniać:

- **Charakter problemu** – np. klasyfikacja binarna vs. wieloklasowa, regresja liniowa vs. nieliniowa.
- **Wrażliwość na wartości odstające** – MSE jest silnie wrażliwe na anomalie, MAE bardziej odporne.
- **Interpretowalność** – niektóre funkcje, takie jak log-loss, mają bezpośrednie powiązanie z teorią prawdopodobieństwa.
- **Stabilność numeryczna** – przy dużej liczbie klas lub ekstremalnych wartościach wyjściowych warto stosować wersje funkcji kosztu zoptymalizowane pod kątem obliczeń numerycznych.

Ostateczny wybór funkcji straty ma istotny wpływ na przebieg procesu uczenia, tempo konwergencji oraz końcową jakość modelu.

2.5 Algorytmy optymalizacji

Kluczowym elementem procesu uczenia są Algorytmy optymalizacji. Odpowiadają one za aktualizację parametrów modelu w celu minimalizacji wartości funkcji straty. W praktyce optymalizacja odbywa się iteracyjnie, a parametry modelu są aktualizowane zgodnie z kierunkiem przeciwnym do gradientu funkcji kosztu względem wag. [Goodfellow-et-al-2016]

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

gdzie η oznacza współczynnik uczenia (learning rate). Zbyt duża wartość η może powodować niestabilność uczenia, a zbyt mała – spowolnienie konwergencji. Dlatego opracowano szereg metod adaptacyjnych i usprawnień klasycznego spadku gradientu.

Metoda najszybszego spadku

Podstawowy algorytm optymalizacji, stosowany zarówno w wersji pełnej (batch gradient descent), jak i stochastycznej (stochastic gradient descent, SGD). W przypadku danych dzielonych na partie (mini-batches), aktualizacja parametrów przyjmuje postać:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L_i(\theta_t)$$

gdzie m to liczba próbek w partii danych.

W praktyce metoda spadku gradientu występuje w trzech głównych wariantach, różniących się sposobem obliczania gradientu. W pełnym spadku gradientu (batch gradient descent) gradient wyznacza się na podstawie całego zbioru treningowego, co daje dokładny kierunek spadku, ale jest obliczeniowo kosztowne i mało praktyczne przy dużych danych. W przeciwieństwie do tego, stochastyczny spadek gradientu (stochastic gradient descent, SGD) aktualizuje wagi po każdej próbce danych, co znacząco przyspiesza uczenie i umożliwia ucieczkę z minimów lokalnych, jednak wprowadza losowe oscylacje w kierunku gradientu. Kompromisem pomiędzy tymi podejściami jest mini-batch gradient descent, który wykorzystuje niewielkie partie danych (zazwyczaj od kilkunastu do kilkuset przykładów). Takie podejście poprawia stabilność uczenia i umożliwia efektywne wykorzystanie równoległości obliczeń, np. na procesorach GPU.

Choć klasyczna metoda spadku gradientu jest intuicyjna i prosta w implementacji, ma istotne ograniczenia. Jest bardzo wrażliwa na dobór współczynnika uczenia i może łatwo utknąć w minimach lokalnych lub punktach siodłowych. Ponadto w obszarach o silnie nachylonych dolinach funkcji kosztu często występują oscylacje, które spowalniają konwergencję

Spadek gradientu z momentem (Momentum)

Metoda *momentum* to usprawnienie klasycznego spadku gradientu, które wprowadza do aktualizacji wag składnik bezwładności (momentum) odpowiednio skalowany przez parametr γ . Pozwala to na użycie większego współczynnika uczenia. Tempo konwergencji znacząco wzrasta, w szczególności w trudnych obszarach funkcji kosztu z dużą ilością minimów lokalnych.

$$\begin{aligned}1 &= \gamma + \eta \\ v_t &= \gamma v_{t-1} + \eta \nabla_{\theta} L(\theta_t), \\ \theta_{t+1} &= \theta_t - v_t.\end{aligned}$$

- η — learning rate (wielkość kroku).
- $\gamma \in [0, 1)$ — momentum coefficient, określa stopień zachowania poprzednich gradientów.

Efektom jest filtr wygładzający historię gradientów, co przyspiesza konwergencję i redukuje oscylacje, a także ułatwia przeciskanie przez lokalne minima.

Metoda przyspieszonego gradientu

Metoda przyspieszonego gradientu, nazywana metodą Nestorova od nazwiska autora, jest dalszym usprawnieniem metody z momentem, w którym gradient jest obliczany nie w bieżącym położeniu parametrów, a w przewidywanym „przyszłym” położeniu modelu. Dzięki temu aktualizacja jest bardziej precyzyjna i stabilna, co pozwala uniknąć nadmiernego przeskoczenia minimum [Nesterov1983]

$$\begin{aligned}v_t &= \gamma v_{t-1} + \eta \nabla_{\theta} L(\theta_t - \gamma v_{t-1}), \\ \theta_{t+1} &= \theta_t - v_t.\end{aligned}$$

W praktyce metoda Nesterova jest bardziej stabilna od klasycznego momentum i często prowadzi do szybszej konwergencji, zwłaszcza w sieciach głębokich

RMSPProp (Root Mean Square Propagation)

RMSPProp to metoda optymalizacji z adaptacyjnymi współczynnikami uczenia. Jest ona modyfikacją optymalizatora AdaGrad(Adaptive Gradient). Algorytmy tego typu dostosowują tempo uczenia dla każdego parametru indywidualnie. [Hinton2012, MediumRMSPProp].

Aktualizacje w RMSPProp zapisuje się jako:

$$\begin{aligned}V_t &= \beta V_{t-1} + (1 - \beta) (\nabla_{\theta} L(\theta_t))^2, \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{V_t} + \epsilon} \nabla_{\theta} L(\theta_t),\end{aligned}$$

gdzie:

- β (typowo ok. 0.9) — współczynnik wygładzania,
- ϵ — mała stała dla stabilności numerycznej i uniknięcia dzielenia przez zero

- η — współczynnik uczenia.

Dzięki temu zwiększamy tempo nauki parametrów, których gradient jest mały oraz zwalniamy proces nauki parametrów z porównywalnie stromym gradientem. Chroni to sieć przed pominięciem optimum w regionie o stromym gradientie oraz przyspiesza proces nauki w przeciwnym przypadku.

Zalety RMSProp:

- Utrzymuje adaptacyjny krok uczenia bez jego nadmiernej redukcji,
- Poprawia zbieżność w problemach o zmiennych gradientach — często stosowany w RNN.

Wady:

- Wymaga starannego doboru hiperparametrów,
- Może być mniej skuteczny w przypadku ekstremalnie rzadkich gradientów — wtedy lepszy jest Adam.

Adam (Adaptive Moment Estimation)

Adam stara się łączyć zalety optymalizatorów momentum i RMSProp. Uwzględnia kierunek zmian gradientu i wygładza aktualizacje tak jak momentum oraz adaptacyjnie skaluje krok uczenia względem wariancji gradientów jak RMSProp. Z tego powodu jest on obecnie, wraz z wariantami, najpopularniejszym optymalizatorem sieci neuronowych.

W Adamie dla każdego parametru θ śledzi się dwa momenty:

Pierwszy moment, średnia gradientów

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Drugi moment, średnia kwadratów gradientów

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Ze względu na początkowe wartości zerowe m_t oraz v_t wprowadza się poprawki biasu. Dzięki nim na początku uczenie nie jest spowolnione.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Finalna aktualizacja wag ma postać

$$\theta_{t+1} = \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

gdzie $g_t = \nabla_{\theta} L(\theta_t)$, β_1 , β_2 są współczynnikami wygładzania (np. 0.9 i 0.999), a ϵ to mała wartość stabilizująca.

Zalety:

- Adaptacyjny krok dla każdego parametru,
- Efektywny obliczeniowo i pamięciowo,
- Działa dobrze przy danych z szumem, gradientach niestabilnych.

Wady:

- Możliwe problemy ze zbieżnością w funkcjach wypukłych,
- Czasem gorsza generalizacja niż klasyczne SGD z momentum,
- Wymaga dostrojenia wielu hiperparametrów.

[kingma2015adam]

2.6 Problemy gradientowe i usprawnienia

Wraz ze wzrostem głębokości sieci neuronowych proces uczenia staje się coraz bardziej podatny na problemy związane z propagacją błędów i stabilnością optymalizacji. Kluczowym czynnikiem wpływającym na skuteczność uczenia jest sposób, w jaki gradienty funkcji kosztu rozchodzą się przez kolejne warstwy modelu. Jeżeli wartości te zanikają lub eksplodują, uczenie głębokiej sieci staje się nieefektywne lub wręcz niemożliwe. Poniżej przedstawiono najważniejsze zjawiska i techniki ich łagodzenia

Nasycenie

Zjawisko nasycenia występuje, gdy dla dużych wartości wejścia funkcja aktywacji osiąga swoje asymptotyczne maksimum lub minimum, a jej pochodna staje się bliska zeru. W takich warstwach gradient błędów praktycznie nie przepływa dalej, co prowadzi do spowolnienia lub całkowitego zatrzymania procesu uczenia.

Typowym przykładem są funkcje sigmoidalne i tangens hiperboliczny, które dla dużych wartości $|x|$ przestają reagować na zmiany sygnału wejściowego. Wprowadzenie funkcji ReLU i jej wariantów rozwiązało ten problem dla dodatnich wartości argumentu, choć nadal może prowadzić do tzw. martwych neuronów – przypadków, w których gradient po stronie ujemnej jest stale równy zero.

Dodatkowe techniki, takie jak Leaky ReLU czy ELU, wprowadzają niewielkie nachylenie w obszarze ujemnym, co pozwala zachować przepływ gradientu i utrzymać aktywność neuronów w całej sieci.

Skośność rozkładu aktywacji

Kolejnym problemem, często pomijanym, jest skośność rozkładu aktywacji (ang. bias shift). Występuje on, gdy funkcje aktywacji generują wyłącznie dodatnie wartości, jak w przypadku sigmoidy, której zakres wynosi $(0, 1)$. W takiej sytuacji średnia wartość aktywacji jest dodatnia, co powoduje przesunięcie rozkładu wejść kolejnych warstw w kierunku dodatnich wartości. W efekcie sieć pracuje w obszarze nasycenia funkcji aktywacji, a gradienty stopniowo zanikają.

Skośność rozkładu aktywacji utrudnia także utrzymanie stabilnego przepływu sygnału i powoduje, że wagi w kolejnych warstwach są zmuszane do kompensowania przesunięć statystycznych. Rozwią-

zaniem tego problemu jest stosowanie funkcji aktywacji o zakresie symetrycznym względem zera, takich jak $\tanh(x)$, lub metod normalizacji danych, które utrzymują średnią aktywacji blisko zera (np. Batch Normalization).

Zanikanie gradientu

Zjawisko zanikającego gradientu polega na tym, że wartości gradientów w trakcie propagacji wstecznej stają się coraz mniejsze w miarę przechodzenia przez kolejne warstwy. W konsekwencji wcześniejsze warstwy sieci uczą się bardzo wolno, podczas gdy końcowe – położone bliżej wyjścia – aktualizują swoje wagi znacznie szybciej. Problem ten wynika głównie z własności funkcji aktywacji, takich jak sigmoida lub tangens hiperboliczny, których pochodne przyjmują wartości bliskie zeru dla dużych wartości argumentu. Dla przykładu, w przypadku tangensa hiperbolicznego:

$$\tanh'(x) = 1 - \tanh^2(x)$$

co oznacza, że dla $|x| > 3$ pochodna staje się praktycznie zerowa. Jeżeli gradient pochodnej jest mniejszy niż jeden, a sieć zawiera wiele warstw, iloczyn kolejnych pochodnych szybko dąży do zera, przez co sygnał błędu praktycznie nie dociera do wcześniejszych warstw.

Aby przeciwdziałać zanikaniu gradientu, wprowadza się funkcje aktywacji, które nie nasycają się dla dodatnich wartości argumentu, takie jak ReLU i jej warianty (LeakyReLU, PReLU, ELU). Dodatkowo stosuje się odpowiednie metody inicjalizacji wag (np. He lub Xavier) oraz techniki normalizacji, które utrzymują rozkład aktywacji w kontrolowanym zakresie.

Eksplodujący gradient

Przeciwnym zjawiskiem jest eksplodujący gradient, który występuje wtedy, gdy wartości gradientów w trakcie propagacji wstecznej stają się coraz większe, często osiągając wartości prowadzące do niestabilności numerycznej. W takim przypadku wagi sieci zaczynają przyjmować bardzo duże wartości, co skutkuje rozbieżnością funkcji kosztu i przerwaniem procesu uczenia. Problem ten występuje szczególnie często w głębokich sieciach rekurencyjnych oraz w modelach o źle dobranych współczynnikach uczenia.

Jednym z najczęściej stosowanych sposobów zapobiegania eksplozji gradientów jest ograniczanie ich normy (ang. gradient clipping), które polega na skalowaniu wektora gradientu, jeśli jego długość przekracza ustalony próg τ :

$$g \leftarrow \frac{g}{\max\left(1, \frac{\|g\|}{\tau}\right)}$$

gdzie:

- g — wektor gradientu
- $\|g\|$ - jego norma (najczęściej euklidesowa, czyli 2)

- τ - ustalony próg (np. 1.0)

Rozdział 3

Sieci Konwolucyjne

3.1 Geneza

Sieci splotowe (ang. Convolutional Neural Networks, CNN) stanowią jedną z najważniejszych klas architektur głębokiego uczenia. Ich powstanie inspirowane było budową kory wzrokowej, w której poszczególne neurony reagują na proste wzorce w ograniczonych fragmentach pola widzenia. W odróżnieniu od klasycznych perceptronów wielowarstwowych, w CNN każdy neuron przetwarza tylko lokalny obszar danych wejściowych co pozwala sieci na wykrywanie wzorców przestrzennych takich jak krawędzie, linie, tekstury czy bardziej złożone obiekty.

Takie podejście sprawia, że CNN szczególnie dobrze nadają się do analizy danych przestrzennych, takich jak obrazy, mapy czy stany gry. Filtry konwolucyjne działają niezależnie od położenia wzorca, co oznacza, że ta sama cecha (np. zawodnik na planszy gry BloodBowl) może być rozpoznana w różnych miejscach bez potrzeby uczenia oddzielnych wag. Dodatkowo mechanizm współdzielenia wag ogranicza liczbę parametrów modelu, co zmniejsza ryzyko przeuczenia i przyspiesza uczenie.

W kontekście gier o dużej złożoności, takich jak BloodBowl, sieci konwolucyjne pełnią rolę ekstraktora cech przestrzennych, przekształcając planszę gry w hierarchię reprezentacji umożliwiającą agentowi lepsze zrozumienie sytuacji na boisku. Tego typu architektury są powszechnie stosowane w algorytmach uczenia ze wzmocnieniem, takich jak A2C czy PPO, gdzie służą do przetwarzania obrazu stanu gry przed podjęciem decyzji przez agenta. [lecun1998gradient]

3.2 Sposób działania

Warstwa konwolucyjna jest oparta na operacji splotu. Polega ona na obliczeniu sumy ważonej wartości lokalnych. Dzięki temu warstwa splotowa umożliwia wykrywanie charakterystycznych wzorców, takich jak krawędzie, niezależnie od ich położenia.

$$y(i) = (x * w)(i) = \sum_{m \in \mathcal{M}} x(i + m) w(m),$$

- i - wektor indeksów wyjścia
- m - wektor przesunięć w przestrzeni jądra
- $x(i + m)$ - fragment danych wejściowych
- $w(m)$ - wagi jądra
- $y(i)$ - wartości wyjściowe w punkcie i

Z racji, że jak w większości gier planszowych, gra bloodbowl odbywa się na 2 wymiarowej planszy, najprzydatniejszym typem filtra będzie typ 2 wymiarowy.

$$y(i, j) = (x * w)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x(i + m, j + n) w(m, n),$$

Filtr przesuwany jest po danych wejściowych z określonym krokiem (stride). W efekcie powstaje tzw. mapa cech (feature map), która wskazuje miejsca występowania danego wzorca w danych.

Struktura warstwy konwolucyjnej

Każda warstwa konwolucyjna składa się z:

- zestawu filtrów (jąder) – niewielkich macierzy (np. 3×3 , 5×5), które uczą się rozpoznawać określone cechy obrazu lub stanu gry
- operacji przesuwania filtra (stride) – definiuje, co ile pól filtr jest przesuwany w poziomie i pionie
- paddingu – uzupełniania brzegów wejścia zerami lub inną wartością, by kontrolować rozmiar wyjścia,

Kluczową właściwością CNN jest współdzielenie wag, dzięki czemu jeden filtr działa w całym obrazie w identyczny sposób. Pozwala to drastycznie ograniczyć liczbę parametrów w porównaniu z siecią w pełni połączoną.

Mapy cech i hierarchia reprezentacji

Każdy filtr w warstwie konwolucyjnej uczy się wykrywać określony wzorec np pola przez które przechodząc zawodnik może zostać zatrzymany.

Kolejne warstwy CNN tworzą hierarchię reprezentacji:

- pierwsze warstwy rozpoznają proste cechy lokalne
- środkowe warstwy łączą proste cechy w bardziej złożone struktury
- ostatnie warstwy wydobywają informacje semantyczne - np. pola o niskim ryzyku negatywnych efektów

Wynik działania ostatnich warstw konwolucyjnych jest często spłaszczany (flattening) i przekazywany do klasycznych warstw gęstych (fully connected), które dokonują końcowej klasyfikacji lub decyzji.

3.2.1 Warstwy wspierające

Aby zwiększyć efektywność uczenia i zdolność generalizacji, w sieciach konwolucyjnych stosuje się dodatkowe warstwy

- Pooling - redukuje rozdzielczość map cech poprzez agregację lokalnych fragmentów (np. 2×2).
- Batch Normalization – normalizuje wartości aktywacji w obrębie mini-batcha, stabilizując proces uczenia i przyspieszając zbieżność.
- Dropout – losowo „wyłącza” część neuronów w czasie treningu, co zapobiega przeuczeniu (overfitting)

Kombinacja konwolucji, poolingów i nieliniowych aktywacji pozwala sieci tworzyć wielowarstwowe reprezentacje danych przy zachowaniu wysokiej efektywności obliczeniowej.

Rozdział 4

Środowisko testowe

W celu porównania rozwiązań uczenia ze wzmocnieniem, użyję jako środowiska testowego gry BloodBowl. Jest to gra planszowa, oparta na zasadach footballu amerykańskiego. Każdy gracz kieruje jedną z dwóch drużyn składających się z 11 zawodników. Istnieje wersja o mniejszym rozmiarze drużyn oraz boisku. Gra jest podzielona na dwie połowy, drużyny na zmianę starają się,



4.1 Just in case

Wykaz skrótów i symboli

Spis rysunków

1	Porównanie płytkiej i głębokiej sieci neuronowej z wyrównanym położeniem neuronów wyjściowych.	16
---	--	----

Spis tabel